NEW Announcing our new AI engineering course: **How LLMs Work**

Roadmaps AI Tutor Upgrade Login Sign Up

8 In-Demand Backend Developer Skills to Master



Fernando Doglio

Prefer us on Google

Whether your goal is to become a backend developer or to stay relevant as udent mindset. The ever- uous learning, regardless eworks, libraries, and t solutions to old per, you'll have to stay

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

Customise Reject All Accept All

In this Article

Understanding Backend Development
Proficiency in Core and Emerging Programming Languages
Software Design and Architecture
Mastery of Database Management Systems
API Development Capabilities
Version Control Savvy
Testing
CI/CD and DevOps Familiarity
Soft Skills
Conclusion

In this article, we'll cover the following set of backend developer skills we recommend you aim for:

- Keeping an eye on core and new backend programming languages
- Understanding the basics of software design and architecture
- Understanding databases and how to use them
- API development

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

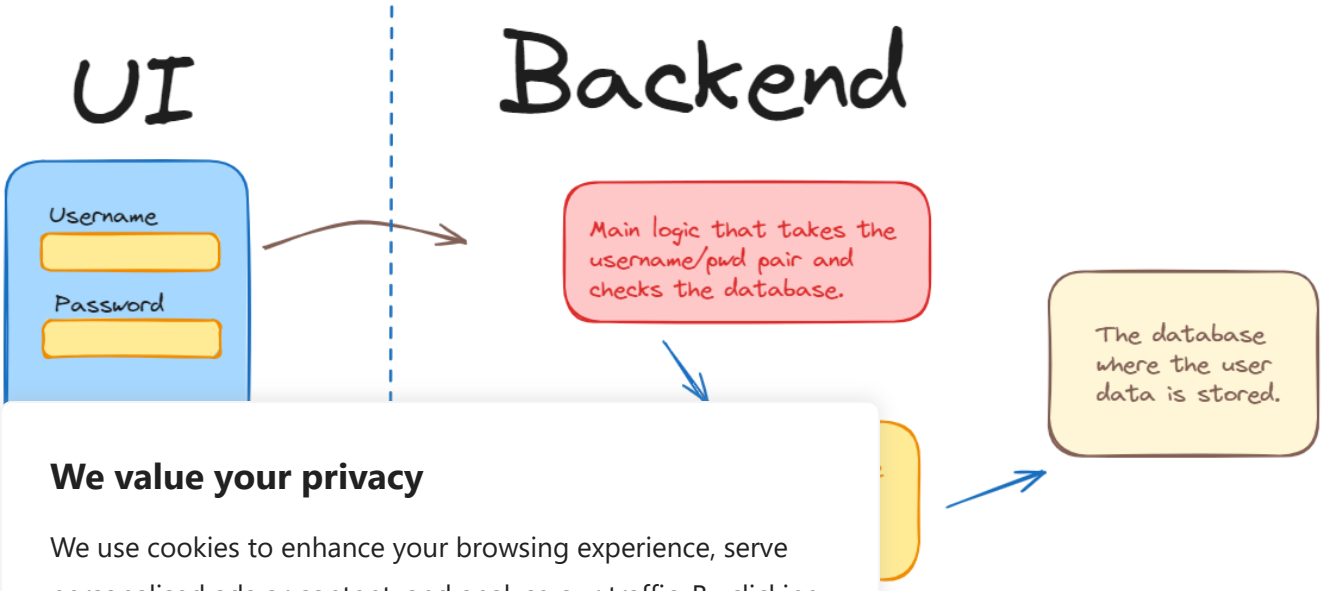
So, let's get going!

Understanding Backend Development

Before we move on and start discussing the different backend development skills you should focus on, let's first understand what a backend developer is. After all, if you're looking to actually become a backend developer, you'll need this.

A backend developer focuses entirely on writing business logic for an application and much of the supporting logic as well.

That said, there might be applications where the business logic is split into the frontend and the backend. However, while the frontend dev might have to share their time between UI code and business logic, the backend dev will focus most of their time on core business logic. That's the main difference between the two.



We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

more behind the curtain

. In this case, a "simple"

usiness logic.

Let's now look at the most in-demand backend developer skills you should focus on in backend development.

Proficiency in Core and Emerging Programming Languages

One of the most basic skills you should focus on as a backend developer is on identifying key programming languages to learn (or at least keep an eye out for).

There are some essential backend languages that the industry has adopted as de facto standards. This means most new projects are usually coded using one (or multiple) of these programming languages.



We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

- are:

- **JavaScript (or any of its variants, such as TypeScript).** This is a very common option because it's also the language used by frontend developers, thus making it easier for developers to work on both sides of the same project.
- **Python.** While a very common option for other types of projects (such as data processing and **data science**), it's still very popular in the web development world. Python has many good qualities and supporting frameworks that make it a very easy-to-pick-up option for coding backend systems.
- **Go (A.K.A Golang).** This programming language was developed by Google. It was designed with simplicity, efficiency, and concurrency in mind. That's made it gain popularity in the backend development space, making it an interesting option for projects that prioritize performance and concurrency.
- **Java.** One of the most common alternatives for enterprise solutions, Java, has been constantly evolving since its first release back in 1995. All that time making its way into big enterprises that trust its robustness and ever-growing community of developers. While not the easiest language to learn, it's definitely up there in the top 10 most

ckOverflow's 2023

We value your privacy

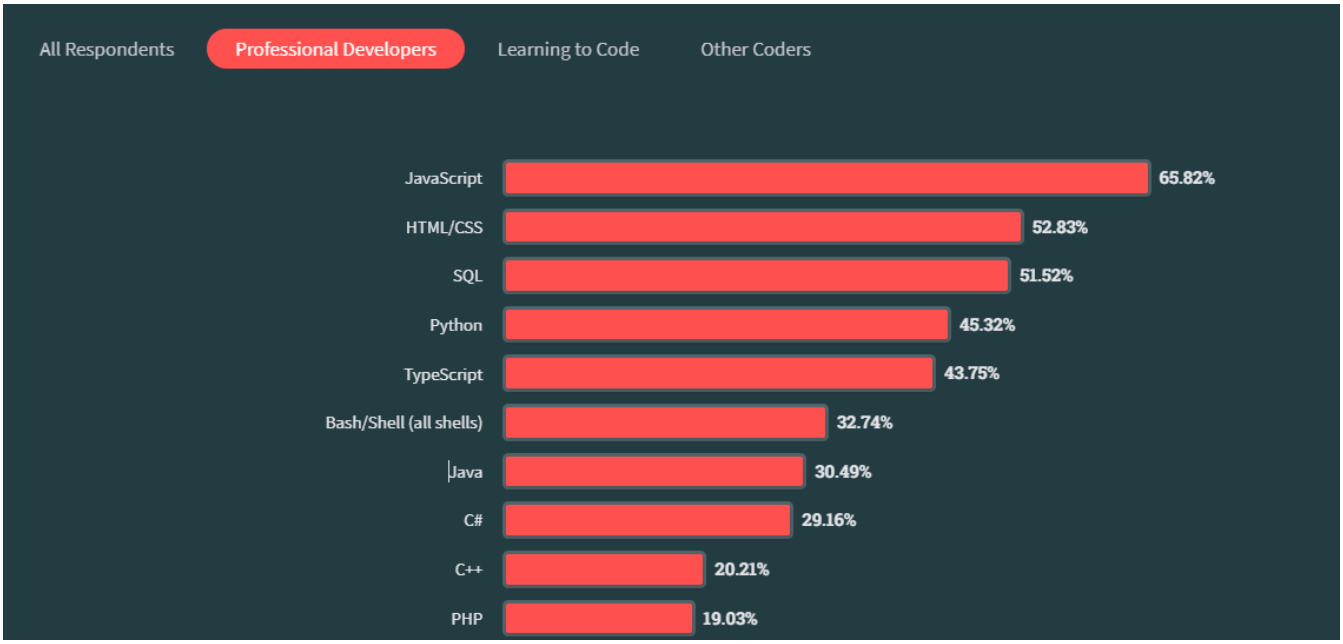
We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

l above, from the

ant languages to pay

es amongst professional

developers (screenshot taken from SO's survey of 2023):



Keeping an eye on the rising stars

If working with at least one of the most common backend languages was important, understanding what are the rising technologies in the backend world is just as crucial.

You won't see a new programming language being released every month. However, in the span of a few years, you might see the release of several, and out of those, some might stick long enough to become new standards.

For example, take a look at the period between 2012 and 2015; in just 3 years, 9 programming languages were released, out of which most of them

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.



- In 2012, we got Julia, Elm, Go, and TypeScript.
- In 2013, we got Dart
- In 2014, we got Swift, Hack, and Crystal
- And in 2015, we got Rust.

Some of those languages are very relevant to this day, such as TypeScript and Rust, while others, such as Hack and Crystal, might be known to only a few in very niche sectors.

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

(mming language will
l ed to hone is that of
l merging trends.

orks

Frameworks for a specific programming language do change a lot faster than the language itself, though.

Frameworks are there to provide you with a simplified gateway into the functionalities that you'd normally need for common tasks. For example, in the context of backend web development, frameworks usually take care of:

- **Parsing HTTP requests** and turning them into objects you can easily interact with (so you don't have to learn how the HTTP protocol works).
- **Abstracting concepts,** such as a request or a response, into objects and functions that you can reason about at a higher level. This gives you an easier time thinking about how to solve a problem using these tools.
- **Accessing data becomes a lot easier when there are abstractions.** Some frameworks provide what is known as an ORM (Object Relational Mapping). Through ORM, you can interact with databases without having to think about writing SQL queries or even database schemas.
- And many more.

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

u'll do as a backend
em. Of course, learning
impossible. Instead,
nat are the most
f them.

Software Design and Architecture

Coding is not just about writing code.

While that statement might be a bit confusing, the truth is there is a lot of architecture involved in software development (both in frontend and backend development). Sometimes, working on these aspects of a system is the job of a specific role called “architect.” However, for backend systems, it’s not uncommon for backend developers to also be involved in architecture conversations and decisions. You’re helping design the underlying backend infrastructure, after all.

The following diagram shows an example of what a very simple system’s architecture might look like:



We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

While the example is oversimplified, it gives you an idea of what the practice of “architecting a system” is.

Essentially, architecting a system means coming up with concepts that represent different aspects of the solution and then deciding how you want to make them interact with each other.

Why is architecture so important here? Because it gives you properties such as code encapsulation, separation of concerns, reusability, and even scalability as a byproduct of the architecture itself.

Let’s take a quick look at some of the most common architectures used for creating backend systems.

Most common backend architectures

There are too many different architectural styles and patterns to cover them all inside a single article, but let's just look at some of the most common ones and how they can help you while working on your backend system.

- **Monolithic architecture:** In a monolithic architecture, the entire application is built as a single, tightly coupled unit. All components (f… …the same codebase. This …y natural to develop …w out of proportion, then …

  …tion is divided into small, …pecific business capability. These services communicate through APIs.

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

- **Service-Oriented Architecture:** Similar to microservices, a service-oriented architecture splits functionality into individual services. The main difference is that these services aren't as granular as a microservice, so they might incorporate functionalities related to different business entities.
- **Event-driven architecture:** With this architecture, each system (or service) responds to events (e.g., user actions and messages from other services) by triggering actions or processes. All services communicate with each other indirectly through an event bus (also known as a "message bus"), so it removes the possibility of having two or more services coupled with each other (meaning that they can't be treated individually).
- **Serverless Architecture:** Also known as Function as a Service (FaaS), serverless architecture allows you to focus on writing code without worrying about the server where they'll run. Functions are executed in response to events without the need for provisioning or managing servers (this is done FOR you automatically).
- **Microkernel architecture:** This architecture lets you build the core, essential functionality into a small microkernel and have the rest of the moved or exchanged

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

and principles mentioned
rchitecture roadmap.

Mastery of Database Management Systems

As a backend developer, you will undoubtedly have to deal with database administration in your daily tasks. They are the industry standard for storing persistent data.

Because of that, it's important to understand that you should be aware of two main categories of databases: SQL databases and NoSQL databases.

SQL databases

These are the standard structured databases (A.K.A relational databases) where you need to define the schema for your data (essentially the data structures you're dealing with), and then you'll use a language called **SQL (Structured Query Language)** to interact with the data inside it. Most backend developers will interact with SQL databases at some point in their career, as this is the most common type of database.

NoSQL databases

As the name implies, these are not your standard SQL databases; in fact, within this category, there are columnar databases, document-based ones (such as **MongoDB**), key-value-based ones (like **Redis**), and more. They don't use predefined data structures, giving you more flexibility and control

kend developers will deal

ferent sub-types, and

here a mongoDB roadmap if you're interested).

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

- Redis, an in-memory key-value pair database.
- Neo4J, a graph database.
- ElasticSearch, a document-based search engine.

In the end, the decision between SQL and NoSQL is about trade-offs and figuring out what works best for your particular use case.

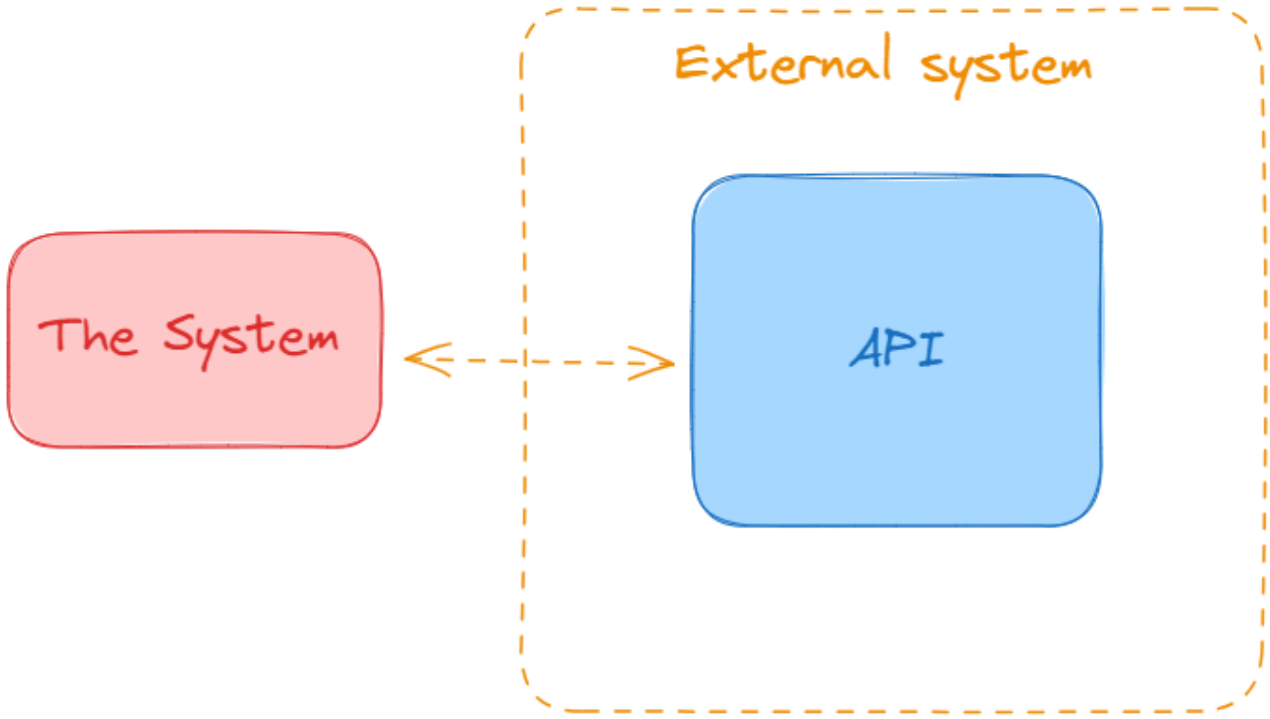
API Development Capabilities

Application Programming Interfaces (APIs) are everywhere. They power the backend of almost all major systems out there (according to a **study conducted by O'Reilly in 2020**, 77% of companies were using microservices/APIs).

That is to say, if you're thinking about becoming a backend developer, you will be coding APIs/microservices. This is why understanding the basics of them is crucial to ensuring your relevance in the field.

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.



The above diagram explains how APIs interact with whatever you might be building.

Now, if you're inside the "**The System**" box, then you need to understand how to interact with these APIs using the right tools. If you're inside the "**External System**" box, then you need to understand the type of standards these APIs need to follow and how to implement them.

Don't worry though, for both situations, there are always frameworks and libraries you can use to simplify your task and ensure you're following the

try currently are REST

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

As a backend developer, it's not mandatory that you master both of these types, but it's definitely recommended that you have some practical experience with one of them.

- **RESTful APIs.** These are APIs that work over HTTP and make extensive use of the HTTP Verbs to give meaning to each request. They've been the most popular type of API until recently, so there are still a lot of projects and teams that make use of it.
- **GraphQL.** GraphQL APIs operate over HTTP as well, leveraging the HTTP protocol and its verbs. In contrast to the conventional RESTful APIs, GraphQL has emerged as a powerful alternative, offering a flexible and efficient approach to data querying and manipulation. GraphQL allows clients to request only the data they need, providing a more tailored and efficient interaction between clients and servers.

Is there one better than the other? There is no easy way to answer that question as both are capable of doing everything you'd need. It's more about your particular requirements and the preferences of your dev team.

Version Control Savvy

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

hould work on (actually,
other words,
trol systems.

sion control tool that
e moment of writing this

is **Git**, while there might be some teams using other (older) tools, as long as you understand the current one, you'll be in good shape.

What is version control?

Version control references the ability for you and other developers to share code with each other while you're working on the same files.

While Git is the industry standard at the moment, GitHub has created such a popular platform around Git, that it almost makes it mandatory to learn about.

So go ahead and create an account, browse what others are doing, and upload your own personal projects. It's definitely a great way to learn.

What should you learn about Git?

If you're picking up Git as your version control system of choice, there are two main areas you should be focusing on.

- **The basics.** Clearly understanding how Git works and the basic commands to add, push and pull changes. You should aim to learn enough about them to feel comfortable using them on your day-to-

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

not enough. While our code, when the enough, the tool alone th **branching strategies**

Keep in mind that Git and Git branching are not trivial topics, and they'll take a while to master. So, while you should give yourself time to learn about them, also make sure you check with others (or use tools such as ChatGPT) to validate your commands before using them. Remember, a wrong Git command or a wrong workflow can cause major problems within a project, especially if there are many developers working on the same codebase.

Testing

Understanding both what testing is and the importance of it within the backend development workflow is crucial for all developers, and one of the mandatory backend developer skills to focus on.

Testing is the development process of making sure your code works in a way that doesn't involve you manually testing every feature but rather using tools that allow you to test and reproduce any problems that can be found programmatically.

This, of course, helps to remove potential human error from the equation when testing big systems and to increase the speed at which these tests

it manually).

describe here. Just know

, and all backend

doing code testing.

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

You'll write tests using a testing framework for every publicly available

function/method in your code. This way you're making sure every piece of code that can be used is tested and performs according to plan. Running these tests is usually quite fast, so you'll be doing it before every commit (usually).

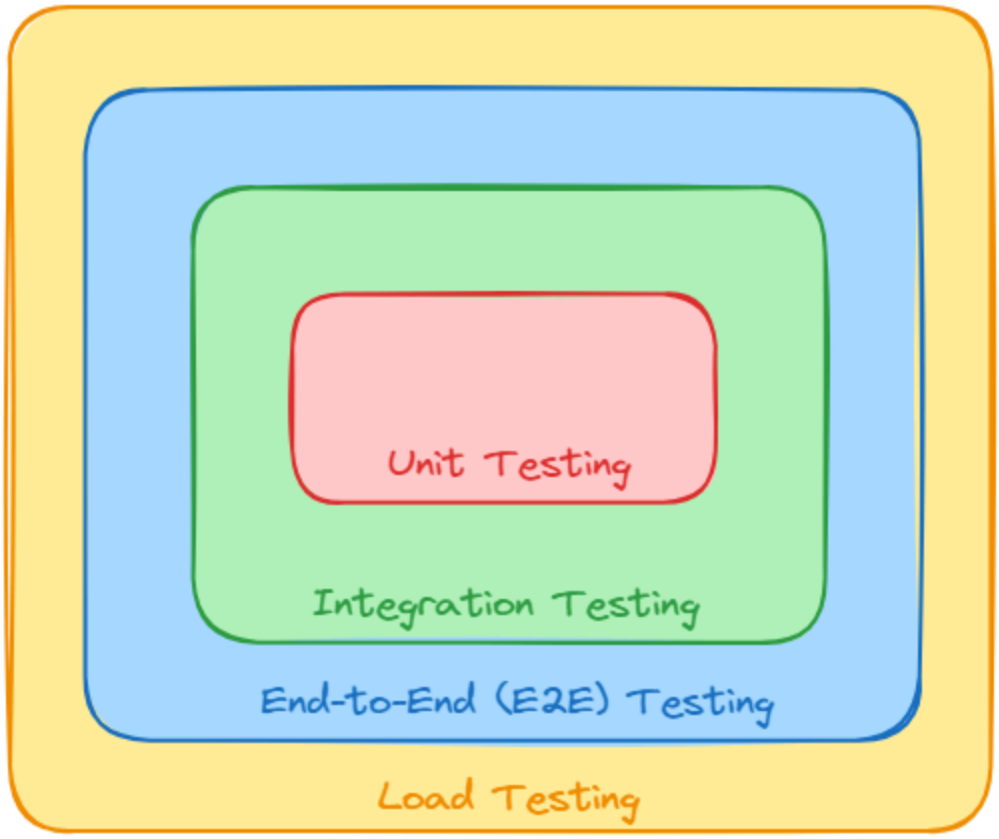
- **Integration testing:** If you're building a system that consists of multiple independent systems working together (think, for instance, a microservice-based architecture), then testing each individual part is not enough. You also have to make sure systems that should interact with each other do so correctly. This is where integration tests come into play.
- **End-to-end testing (E2E):** These tests are similar to integration tests, but they also include the UI of the system. There are tools you can use to automate actions in your UI as if a real user were performing them and then checking the result. For example, clicking on a log-out button and checking if you're later redirected to the log-in screen. This flow would involve the backend performing some actions that result in the user being logged out.
- **Load testing:** While not exactly the same process as with the previous test types, load testing is great for backend systems because it helps

l with high amounts of

following diagram:

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.



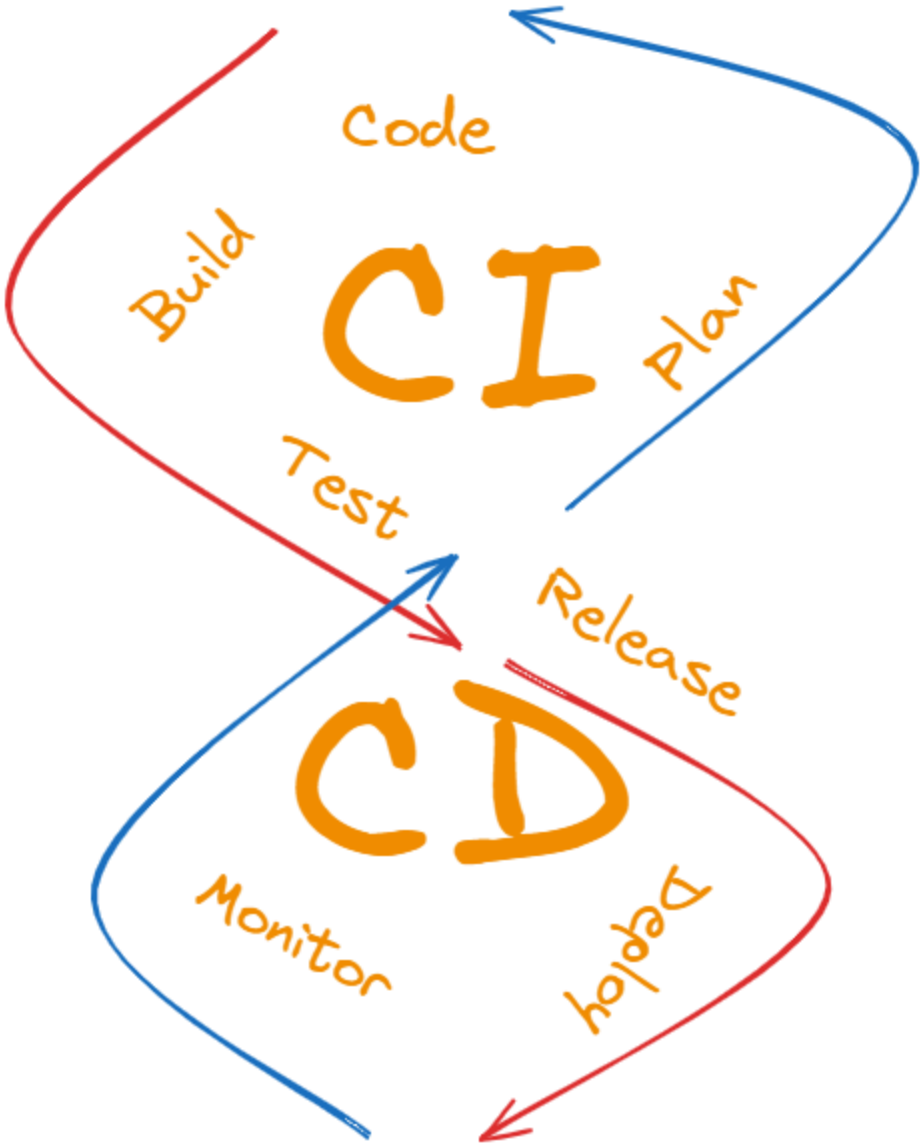
If you're just getting started with testing, I'd recommend focusing only on unit testing for the time being. Once you have a grasp on it, start moving out following the above diagram and slowly move into the other types as you progress.

CI/CD and DevOps Familiarity

As a backend developer, your code will be constantly deployed, either into
on-premise servers. The
(Continuous Integration

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.



These processes will automatically test it (Continuous Integration) and automatically deploy it (if all tests go well). As a backend developer, you're not usually expected to know and understand how to configure these

bout them.

backend developer.

nt be "gently pushed"

ing up servers, and more.

ofessionals with the role of DevOps. Their specialty is automation, making the deployment process

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

efficient and ensuring that everything runs smoothly in the real-world server environment. They play a crucial role in maintaining the reliability and performance of applications and websites.

So, while they're not strictly the responsibilities of backend developers, they're close enough to the role's day-to-day that it would be a good idea to learn about them. If you're interested in learning more about DevOps, check out **our DevOps roadmap** containing all the key topics you should learn about if you want to become a DevOps engineer.

Soft Skills

Finally, the last set of backend developer skills you should focus on are, actually, not technical skills, nor are they exclusively useful for backend developers. These are skills that every developer should work on during their career: soft skills.

Improving communication

The ability to communicate with others, both technical and non-technical people, is crucial in any developer's career.

ecause communicating
han other roles, such as
at they're building.
plain problems or
rm requirement
mprove your own

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

problem-solving skills by being better at articulating the problems and potential solutions to yourself.

Critical thinking

Honing your critical thinking as a backend developer will help your ability to analyze complex problems, identify patterns much faster, and come up with innovative solutions to the problems you're facing.

Pushing the limits of your critical thinking skills will also foster a more systematic and strategic approach to coding and architecting robust and efficient solutions.

In other words, it'll make you a better and more efficient coder. And who doesn't want that?

Conclusion

To summarize, if you expect to become a backend developer or to grow in the area of backend development:

- Keep an eye on the industry to understand what's the status quo and what's new and hot

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

architecture.

abases as well; they're

rt of almost every project

- Remember, testing might look like it's not mandatory, but it's definitely a standard practice when it comes to backend development.
- CI/CD and DevOps are practices you'll be involved with, either directly or indirectly, so learn about them.
- Soft skills are just as important as technical skills if you expect to grow in your career.

That said, do not take this list as the ultimate roadmap but rather as a starting point. If you're willing to take your backend developer career to the next level, push yourself out of your comfort zone and pursue the skills listed here and the ones listed in this detailed **backend roadmap**.

Remember, constant learning is the only absolute truth in the software development world (this is true for backend developers, too). If you keep your skillset updated with the latest trends, you'll remain adaptable and effective as a backend developer.

Related Guides

Types of NoSQL Databases: How to Choose the Right One

Backend Developer Job Description in 2026

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

Join the Community

roadmap.sh is among the **top 10 most starred projects on GitHub** and is visited by hundreds of thousands of developers every month.

Top 10 out of 28M!

371K

GitHub Stars

Star us on GitHub

Help us reach #1

+90k every month

+3.2M

Registered Users

Register yourself

Commit to your growth

+2k every month

52K

Discord Members

Join on Discord

Join the community

Roadmaps Guides FAQs YouTube

roadmap.sh by @nilbuild

Com
arti
pat

© r

TI

The
clou
development and deployment.

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.

DevOps · Kubernetes · Cloud-Native

Cookie Settings

We value your privacy

We use cookies to enhance your browsing experience, serve personalised ads or content, and analyse our traffic. By clicking "Accept All", you consent to our use of cookies.